

OpenJLS

JPEG-LS Lossless Encoder IP Core — Datasheet

Version 1.0 • Single-component, fully pipelined, vendor-agnostic VHDL

1 Overview

OpenJLS is a JPEG-LS lossless encoder for FPGAs. It encodes single-component (grayscale) images at one pixel per clock with no external memory, streaming a standards-compliant JPEG-LS bitstream out over a standard ready/valid streaming handshake. Source code, license and the full verification suite are public at <https://github.com/VitorMendesC/OpenJLS>.

Table 1: Key specifications.

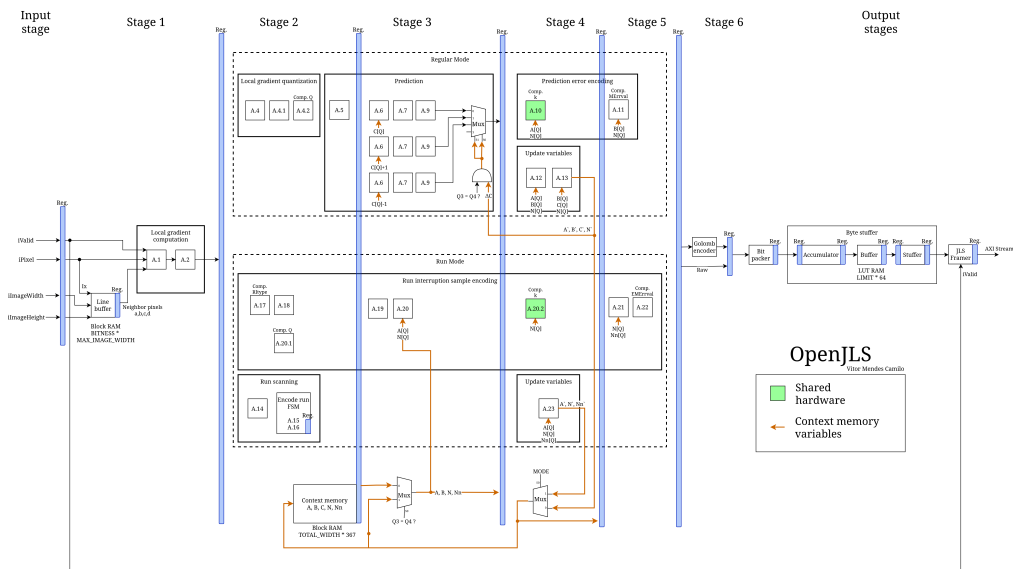
Standard	JPEG-LS (ISO/IEC 14495-1, ITU-T T.87)
Compression	Lossless only
Components	Single (grayscale)
Pixel depth	8–16 bits (BITNESS)
Image size	Up to 65535×65535 px
Throughput	1 pixel / clock
Latency	14 clock cycles
f_{max}	~240 MHz (Xilinx xczu7eg-1)
Interface	Ready/valid streaming (AXI4-Stream / Avalon-ST compatible)
Memory	On-chip line buffer, one image row; no external RAM
Resources	~8k LUT, ~2.1k FF + BRAM scaling with image width
Portability	Vendor-agnostic VHDL

2 Functional description

The core is a fully pipelined encoder with a latency of 14 clock cycles: an input register, six processing stages (gradient computation, context modeling, MED prediction with bias correction, Golomb–Rice / run-length coding) and three internally registered output stages (bit packer, byte stuffer, framer).

End-of-image is derived internally from the dimensions provided at the ports `iImageWidth` and `iImageHeight`. The core never stalls — neither at EOI (end of image) nor at EOL (end of line) — which allows processing multiple images back-to-back. The only exception is when the downstream stalls long enough for the byte-stuffer buffer to fill, which in turn stalls the upstream.

Figure 1: Encoder pipeline architecture.



3 Generics

The core is configured by the four generics in Table 2.

Table 2: Configuration generics.

Generic	Range	Description
BITNESS	8–16	Pixel bit depth.
MAX_IMAGE_WIDTH	4–65535	Largest width supported; sets line-buffer depth.
MAX_IMAGE_HEIGHT	1–65535	Largest height supported.
OUT_WIDTH	48–1024	Output bus width in bits; multiple of 8.

MAX_IMAGE_WIDTH and MAX_IMAGE_HEIGHT fix the *compile-time* maximum image size: they size the on-chip line buffer and the dimension counters, so a larger maximum costs more BRAM. They do not select the size of a given image — the dimensions of each encoded image are chosen at *run time* through the `iImageWidth` and `iImageHeight` ports (Table 3), which accept any value from the minimum up to the configured maximum.

4 Ports

All ports are synchronous to the rising edge of `iClk` — the core is a single clock domain. The streaming ports use a plain ready/valid handshake; in Table 3 the **AXIS** column gives the one-to-one AXI4-Stream signal mapping for that ecosystem (Avalon-ST maps the same way at `readyLatency = 0`).

Table 3: Port list and AXI4-Stream mapping.

Port	Dir	Width	AXIS	Role
iClk	in	1	—	Clock.
iRst	in	1	—	Synchronous reset, active high; latches dimensions.
iImageWidth	in	$\lceil \log_2(\text{MAX_IMAGE_WIDTH}+1) \rceil$	—	Image width, pixels (configuration).
iImageHeight	in	$\lceil \log_2(\text{MAX_IMAGE_HEIGHT}+1) \rceil$	—	Image height, pixels (configuration).
iValid	in	1	TVALID	Pixel valid.
iPixel	in	BITNESS	TDATA	Pixel data.
oReady	out	1	TREADY	Input ready.
oData	out	OUT_WIDTH	TDATA	Bitstream beat.
oValid	out	1	TVALID	Output valid.
oKeep	out	OUT_WIDTH/8	TKEEP	Valid-byte mask on final beat.
oLast	out	1	TLAST	End of image.
iReady	in	1	TREADY	Downstream ready.

IMPORTANT:

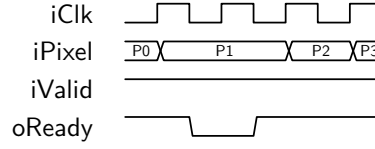
- `iImageWidth` and `iImageHeight` are only sampled while `iRst` is asserted ('1').
- If an invalid value is provided to `iImageWidth`, that is `iImageWidth > MAX_IMAGE_WIDTH` or `iImageWidth < 4`, the encoder uses `MAX_IMAGE_WIDTH` as the image width.
- If an invalid value is provided to `iImageHeight`, that is `iImageHeight > MAX_IMAGE_HEIGHT` or `iImageHeight < 1`, the encoder uses `MAX_IMAGE_HEIGHT` as the image height.
- If you don't intend to change the image dimensions at run-time, wire both inputs to 0 so that the maximum values are used.
- The ports are only $\lceil \log_2(\text{MAX}+1) \rceil$ bits wide, so a value far above the maximum can overflow the port and wrap back into the valid range; the encoder cannot detect this case and won't substitute the maximum.

5 Timing and protocol

Pixel input. Pixels are streamed in scan order — the first row left to right, then the second row, and so on — each an unsigned value on `iPixel`. A pixel transfers on every cycle where `iValid` and `oReady` are both

high. The core asserts `oReady` continuously and deasserts it only when the FIFO in byte stuffer fills under downstream backpressure. Figure 2 shows a transfer with a brief downstream stall.

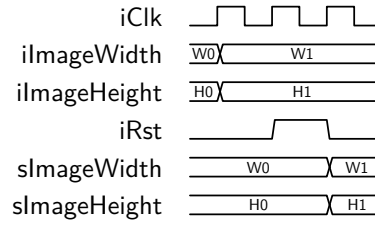
Figure 2: Pixel-input handshake.



In Figure 2, P2 is held flat across the stall and accepted when `oReady` returns high; no pixel is dropped while `oReady` is low.

Reset and configuration. `iRst` is level-sensitive (hold it high for at least one clock) and clears the pipeline. The image dimensions are captured only while `iRst` is high: present `iImageWidth` and `iImageHeight`, hold them stable across the reset pulse, and they latch on the rising edge (Figure 3). A reset is therefore needed only to capture the dimensions — before the first image and whenever the image size changes; with the size unchanged, images encode back-to-back.

Figure 3: Reset and dimension capture.



In Figure 3, the new size (W1/H1) is driven for several cycles, then `iRst` pulses high; the dimensions register on the rising edge while `iRst` is high.

Output. Beats appear on `oData/oValid` subject to `iReady`; `oLast` marks the final beat (EOI) and `oKeep` flags its valid bytes. The bitstream is byte-serial and MSB-first: the earliest byte occupies the most-significant byte of `oData`. Pipeline latency is 14 clock cycles.

6 Resources and performance

Characterized on Xilinx Zynq UltraScale+ `xczu7eg-fbvb900-1-e`, Vivado 2025.2, using the generics `BITNESS = 12` and `OUT_WIDTH = 64`, RTL-only with no floorplanning. f_{max} is the true maximum, found by over-constraining the clock until timing fails. At one pixel per clock, ~ 240 MHz is ~ 240 Mpixel/s.

Figure 4 and Table 4 report f_{max} across `MAX_IMAGE_WIDTH`; Figure 5 and Table 5 report the resource usage.

Figure 4: Maximum frequency vs maximum image width.

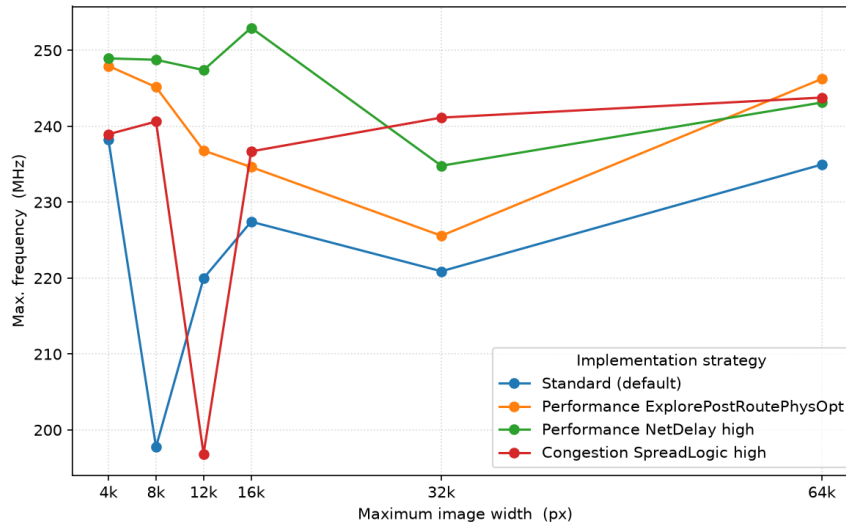
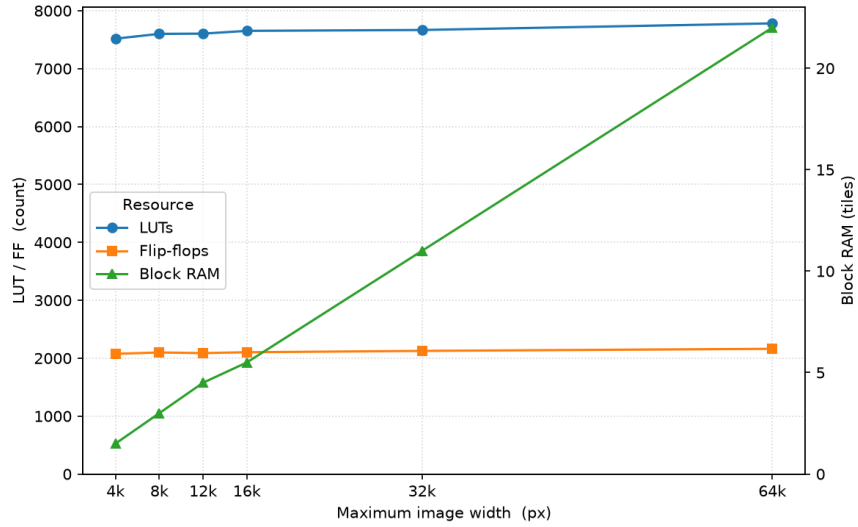


Table 4: Maximum frequency by maximum image width and strategy.

MAX_IMAGE_WIDTH	Default	EPRPO	NetDelay	Congestion
4096	238.2	247.9	248.9	238.9
8192	197.8	245.2	248.8	240.6
12288	220.0	236.8	247.4	196.8
16384	227.4	234.6	253.0	236.7
32768	220.9	225.6	234.8	241.1
65535	235.0	246.2	243.1	243.8

Figure 5: Resource usage vs maximum image width.**Table 5:** Resource usage by maximum image width.

MAX_IMAGE_WIDTH	LUT	FF	BRAM
4096	7517	2075	1.5
8192	7600	2099	3.0
12288	7604	2088	4.5
16384	7652	2103	5.5
32768	7667	2126	11.0
65535	7781	2162	22.0

In Table 4 frequencies are in MHz and the best strategy per row is in bold; the strategies are `Performance_ExplorePostRoutePhysOpt` (EPRPO), `Performance_NetDelay_high` (NetDelay) and `Congestion_SpreadLogic_high` (Congestion). The design is congestion-bound, so the winning strategy shifts with image size; a given netlist is deterministic, but the per-size winner is placement-sensitive and can shift when the netlist changes, so the best-of band (~ 241 – 253 MHz) is the headline figure. Table 5 is for the default strategy; utilization is near-identical across strategies. Logic is essentially constant with image size; only the line-buffer BRAM scales with width and pixel depth.

7 Integration example

```

u_enc : entity work.openjls_top
generic map (
    BITNESS => 8,
    MAX_IMAGE_WIDTH => 4096,
    MAX_IMAGE_HEIGHT => 4096,
    OUT_WIDTH => 64 )
port map (
    iClk => clk,

```

```

iRst => rst,
iImageWidth => img_w, -- latched on reset
iImageHeight => img_h, -- latched on reset
iValid => s_valid, -- pixel stream (AXIS slave)
iPixel => s_data,
oReady => s_ready,
oData => m_tdata, -- bitstream (AXIS master)
oValid => m_tvalid,
oKeep => m_tkeep,
oLast => m_tlast,
iReady => m_tready );

```

Being a single entity with standard ready/valid ports, `openjls_top` can equally be dropped onto a block diagram and wired there, instead of instantiating it in HDL.

8 Conformance and verification

Verified by simulation with NVC using a layered, self-checking suite run at both RTL and post-synthesis (gate-level) stages:

- **OSVVM** — per-module tests against independent T.87-derived reference models, plus top-level control-plane stress (reset injection, backpressure, randomized sizes) with requirements tracking.
- **Coverage** — OSVVM functional coverage with NVC structural code coverage (99%+ statements).
- **Golden model** — byte-exact vs the independent CharLS C++ encoder over 287 real and synthetic images, plus ISO/IEC 14495-1 vectors.
- **Design contracts** — embedded PSL assertions (ready/valid handshake protocol, internal handshakes) on every run.
- **Post-synthesis** — the stress test and a golden-model subset re-run on the synthesized netlist.

Table 6 is the latest regression snapshot (live, per-test reports at <https://vitormendesc.github.io/OpenJLS/>).

Table 6: Latest regression snapshot.

Suite	Status	Test/Cov	Summary
OSVVM suite	PASS	100%	36 tests, 129 810 affirmations (module + top control-plane)
NVC code coverage	info	99.3%	Per-file statement breakdown
Golden model	PASS	100%	287/287 images byte-exact vs CharLS
Post-synth OSVVM	PASS	100%	Control-plane stress on the gate-level netlist
Post-synth golden model	PASS	100%	156/156 images byte-exact vs CharLS

Revision history

Version	Date	Notes
1.0	2026-06	Initial release.

References. ITU-T T.87 / ISO/IEC 14495-1; Y.M. Mert, *Key Architectural Optimizations for Hardware Efficient JPEG-LS Encoder*, IEEE 2018; CharLS; open-logic.